

PROJECT INCREMENT & PROGRESS REPORT

Doton City Pop Discography & Community Vault · Course Code: 6APSI

Project Phase: Midterm / Pre-Public Audit	Date: September 23, 2026
Repository Status: Public Lockdown Prepared	Early Start Status: Initiated Ahead of Schedule

★ EARLY INITIATION ADVANTAGE & STRATEGIC HIGHLIGHT

Proactive Timeline: Work on this project was **started early in the preliminary cycle**, well ahead of standard submission schedules. Initiating early provided significant strategic technical advantages: it allowed ample time for thorough architectural planning, hand-crafting a modular design system (DOTON_DESIGN_SYSTEM.md), integrating a dual dynamic/fallback Supabase data layer, and completing all pre-public security lockdown requirements (SECURITY-CHECKLIST.md) before public repository release.

1. Executive Summary

The **Doton City Pop Vault** platform is a full-stack web application designed to preserve, showcase, and recommend Japanese 1970s and 1980s City Pop vinyl albums. By combining a modern React frontend with a PostgreSQL/Supabase cloud database, the application enables users to filter albums by mood/vibe, play 30-second audio previews continuously across viewports, and submit new community album recommendations in real time.

Due to early commencement of development, all core functional requirements and security milestones have been completed cleanly ahead of deadline. The codebase has undergone comprehensive static and dynamic security audits to guarantee zero credential exposure before flipping the GitHub repository to public status.

2. Incremental Development Milestones

Increment 1: Core Frontend & UI Design System (Completed Early)

- **Modular Design System:** Hand-crafted DOTON_DESIGN_SYSTEM.md using Vanilla CSS variables for high-fidelity dark glassmorphism, gold accents, and multi-theme switching (Night, Day, Sunset).
- **React Component Architecture:** Developed modular components including Header, HeroBanner, FilterBar, AlbumGrid, AlbumDetailModal, and AudioPlayerBar.
- **Persistent Audio Context:** Integrated HTML5 audio player supporting uninterrupted playback while navigating filters and opening detail dialogs, with fallback iTunes preview URL resolution.

Increment 2: Full-Stack Integration & Database Schema (Completed)

- **Relational Database Architecture:** Modeled and deployed albums, tracks, and recommendations schemas on Supabase PostgreSQL with cascade deletion constraints.
- **Hybrid Data Resilience Layer:** Built dynamic REST fetch logic in supabaseClient.js featuring automatic fallback to local seed data (citypopData.js) during network disruptions.

- **Community Recommendations Pipeline:** Implemented reactive submission modal sending new recommendations directly to the Supabase database with optimistic UI updates.

Increment 3: Pre-Public Security Lockdown & Verification (Completed Ahead of Schedule)

- **Git History & Environment Isolation:** Listed `.env` in `.gitignore`, shipped clean `.env.example`, and verified zero secrets in commit logs.
- **Database Access Control:** Enforced Row Level Security (RLS) policies across Supabase tables to safeguard public anon key usage.
- **Security Compliance Verification:** Executed thorough audit and authored `SECURITY-CHECKLIST.md` in the project directory.

3. Pre-Public Security Lockdown Audit Table

The table below details the security verification items completed prior to making the project repository public:

Security Check Item	Status	Verification & Evidence
<code>.env</code> Ignored in Git	Yes	Exposed in <code>.gitignore</code> (lines 14–15); <code>`git status`</code> clean.
<code>.env.example</code> Shipped	Yes	Shipped with placeholder URLs and anon keys only.
No Hardcoded Secrets in Code	Yes	All keys read dynamically via <code>`import.meta.env`</code> ; zero hardcoded keys.
Clean Git Commit History	Yes	Verified via <code>`git log -p`</code> regex search; zero exposed strings.
Supabase Row Level Security (RLS)	Yes	RLS policies enabled on all tables (<code>`albums`</code> , <code>`tracks`</code> , <code>`recs`</code>).
Parameterized Database Queries	Yes	Queries strictly use Supabase JS builder methods (<code>`select()`</code> , <code>`insert()`</code>).
GitHub Actions Workflows	N/A	No <code>`github/workflows`</code> present in repository.
<code>SECURITY-CHECKLIST.md</code> Completed	Yes	Authored in <code>`project/`</code> directory with full row evidence.

4. Engineering & Resource Distribution

Due to starting the project early, development was structured around high manual code craftsmanship paired with strategic AI assistance: **70% Manual Engineering** (hand-crafted CSS design system, state lifting, audio player context, Supabase SQL schema design, RLS security rules) and **30% AI Assistance** (accelerated boilerplate generation, seed script drafting, and initial styling templates).

5. Conclusion & Next Steps

With early commencement and rigorous security lockdown completed, the project repository is fully prepared for public publication and final grading evaluation. Future remaining tasks involve final UI visual checks and hosting deployment.